

Closures → Decorators → Memoization

A concept-first guide with simple examples, code breakdowns and practice questions

Learning path

Closures → Decorators → the @ syntax → building a decorator → memoization → memoized Fibonacci

Core idea: each concept prepares you for the next. The goal is to understand what Python is doing, not just memorise syntax.

1. The Big Picture

These concepts are best learned in this order because each one prepares you for the next. Functions can be stored in variables, passed into other functions and returned from functions. That is the foundation.

The chain to remember

Closure gives an inner function memory. A decorator uses an inner wrapper function to enhance another function. Memoization uses a decorator and a closure to remember previous results.

2. Closures

A closure happens when an inner function remembers a variable from its surrounding outer function, even after the outer function has finished executing.

Start with the simplest example:

```
def outer():  
    message = "Hello"  
  
    def inner():  
        print(message)  
  
    return inner
```

The inner function can access message because Python looks outward when it cannot find the variable inside inner().

Now return the inner function:

```
my_function = outer()  
my_function()
```

Output:

```
Hello
```

Why is this a closure?

outer() has already finished, but inner() still remembers message. The inner function and the remembered surrounding variable form a closure.

A more useful example:

```
def make_adder(n):  
    def add(x):  
        return x + n  
  
    return add  
  
add_5 = make_adder(5)  
add_10 = make_adder(10)  
  
print(add_5(3))  
print(add_10(3))
```

Output:

```
8  
13
```

Here, `add_5` remembers `n = 5`, while `add_10` remembers `n = 10`. The same function pattern creates different closures with different remembered values.

3. Decorators

A decorator is a function that takes another function, adds behaviour to it, and returns a new function.

Start with an ordinary function:

```
def say_hello():  
    print("Hello")
```

Now create a decorator:

```
def decorator(func):  
  
    def wrapper():  
        print("Before")  
        func()  
        print("After")  
  
    return wrapper
```

The decorator receives the original function as func. The inner wrapper adds extra behaviour before and after calling func().

Use the decorator manually first:

```
def say_hello():  
    print("Hello")  
  
say_hello = decorator(say_hello)  
  
say_hello()
```

Output:

```
Before  
Hello  
After
```

Connection to closures

wrapper() uses func from the enclosing decorator() function. Therefore wrapper forms a closure over func.

4. The @decorator Syntax

Python provides shorter syntax for the same wrapping operation:

```
@decorator  
def say_hello():  
    print("Hello")
```

This is essentially equivalent to:

```
def say_hello():  
    print("Hello")  
  
say_hello = decorator(say_hello)
```

Mental model

@decorator means: take the function below, pass it into the decorator, and replace the function name with the function returned by the decorator.

5. Building a Flexible Decorator

Real functions usually accept arguments. A common decorator pattern therefore uses `*args`. For this exercise, the memoization decorator is required to use `*args`.

```
def decorator(func):  
  
    def wrapper(*args):  
        print("Before")  
        result = func(*args)  
        print("After")  
        return result  
  
    return wrapper
```

If the original function receives 5, then `wrapper(5)` gives `args = (5,)`. The tuple can then be unpacked with `func(*args)`.

6. Memoization

Memoization means storing the result of a function call so that the same input can be answered from memory instead of being calculated again.

Think of it as a result notebook:

input → saved result

5 → 5
10 → 55
20 → 6765

Memoization is especially useful for recursive problems such as Fibonacci, where the same values are calculated repeatedly.

6.1 Why Fibonacci needs it

```
fib(5)
├─ fib(4)
│  └─ fib(3)
│     └─ fib(2)
└─ fib(3) ← repeated work
```

Without memoization, these repeated branches cause a large amount of duplicated work as n becomes larger.

6.2 Build the memoize decorator

```
def memoize(func):
    cache = {}

    def wrapper(*args):
        ...

    return wrapper
```

`func` is the original function. `cache` stores previous results. `wrapper` controls whether the result should be retrieved or calculated.

Because `wrapper` uses `cache` from `memoize()`, `wrapper` forms a closure over `cache`.

6.3 Check the cache first

```
def memoize(func):
    cache = {}

    def wrapper(*args):
        if args in cache:
            return cache[args]

        ...

    return wrapper
```

For `fib(5)`, `args` becomes the tuple `(5,)`. So the dictionary can use `(5,)` as its key.

```
fib(5)
  ↓
args = (5,)
  ↓
cache[(5,)]
```

6.4 Calculate, save and return

```
result = func(*args)
cache[args] = result
return result
```

The sequence is simple: call the original function → save the result → return the result.

7. Complete Memoized Fibonacci

```
def memoize(func):
    cache = {}

    def wrapper(*args):
        if args in cache:
            return cache[args]

        result = func(*args)
        cache[args] = result
        return result

    return wrapper

@memoize
def fib(n):
    if n < 2:
        return n

    return fib(n - 1) + fib(n - 2)

print(fib(35))
```

Output:

```
9227465
```

7.1 What @memoize does

```
@memoize
def fib(n):
    ...
```

is essentially:

```
def fib(n):
    ...

fib = memoize(fib)
```

After decoration, the name `fib` refers to the wrapper. The wrapper still has access to the original Fibonacci function through `func`, and it remembers the cache through its closure.

7.2 What happens during a call?

```
fib(5)
↓
wrapper(5)
↓
args = (5,)
↓
Is (5,) in cache?
├─ Yes → return saved answer
└─ No
    ↓
    calculate using func(*args)
    ↓
    cache[(5,)] = result
    ↓
    return result
```

Final mental model

Closure remembers. Decorator wraps. Memoization remembers results.

8. Concept Summary

Concept	What it means
Closure	An inner function remembers variables from its surrounding function.
Higher-order function	A function that takes functions as arguments and/or returns a function.
Decorator	A function that receives another function and returns an enhanced function.
@decorator	Short syntax for replacing a function with the function returned by the decorator.
Memoization	Caching function results so repeated inputs do not require repeated calculations.

9. Practice Questions

1. Why can an inner function still use a variable after the outer function has finished?
2. In `make_adder(5)`, what value does the returned function remember?
3. What is the difference between returning `wrapper` and calling `wrapper()`?
4. What does `@decorator` mean in ordinary Python syntax?
5. If `wrapper(7)` uses `*args`, what is `args`?
6. Why can `args` be used as a dictionary key here?
7. Where is cache stored, and why is it not global?
8. What happens when the same Fibonacci input is requested again?

Mathias.nerd

Python problem-solving notes